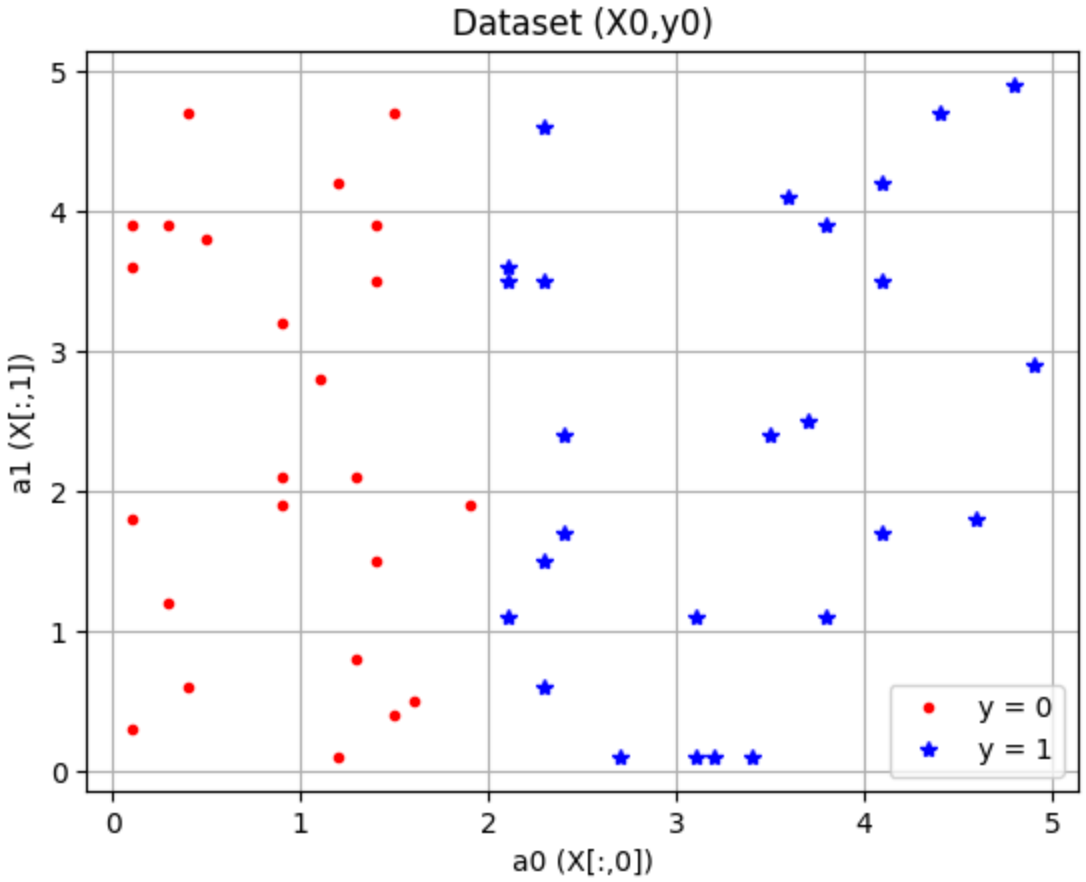
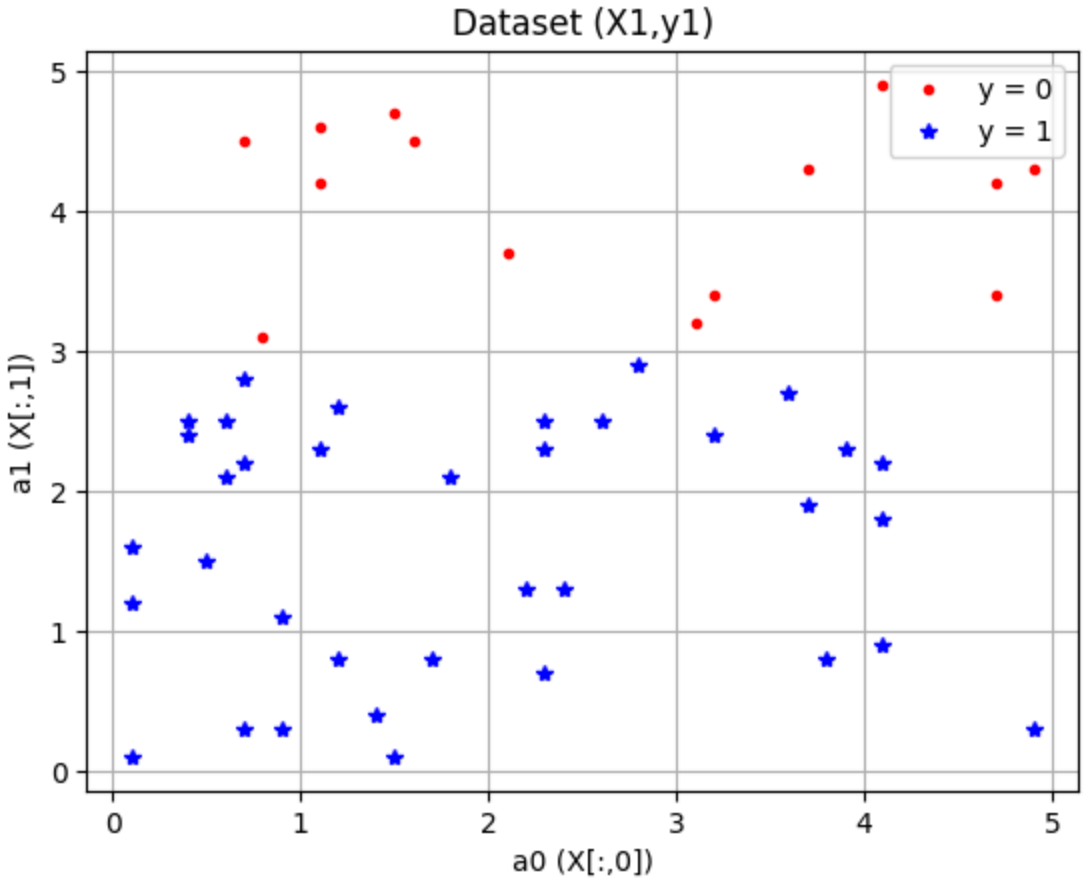
In [12]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
```

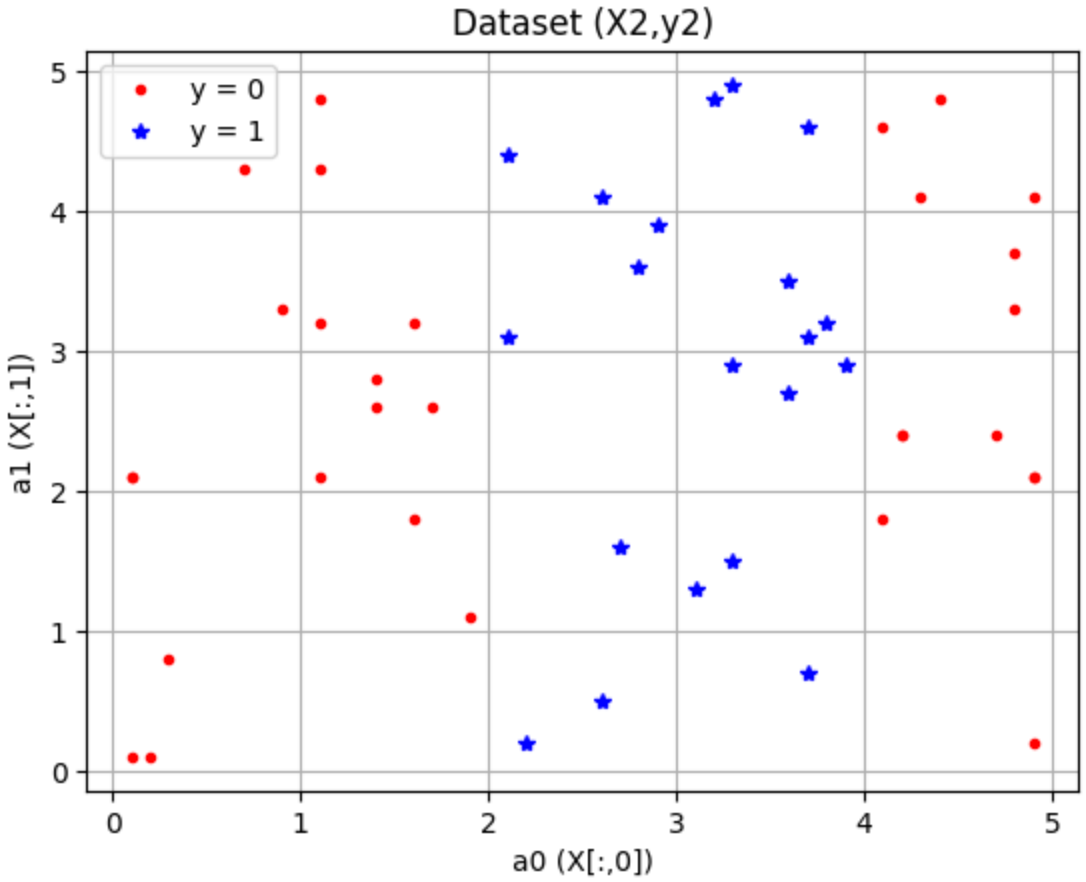
In [13]:
```
data_dir = 'Datasets_xy'
print(f"Using data from {data_dir}")
```

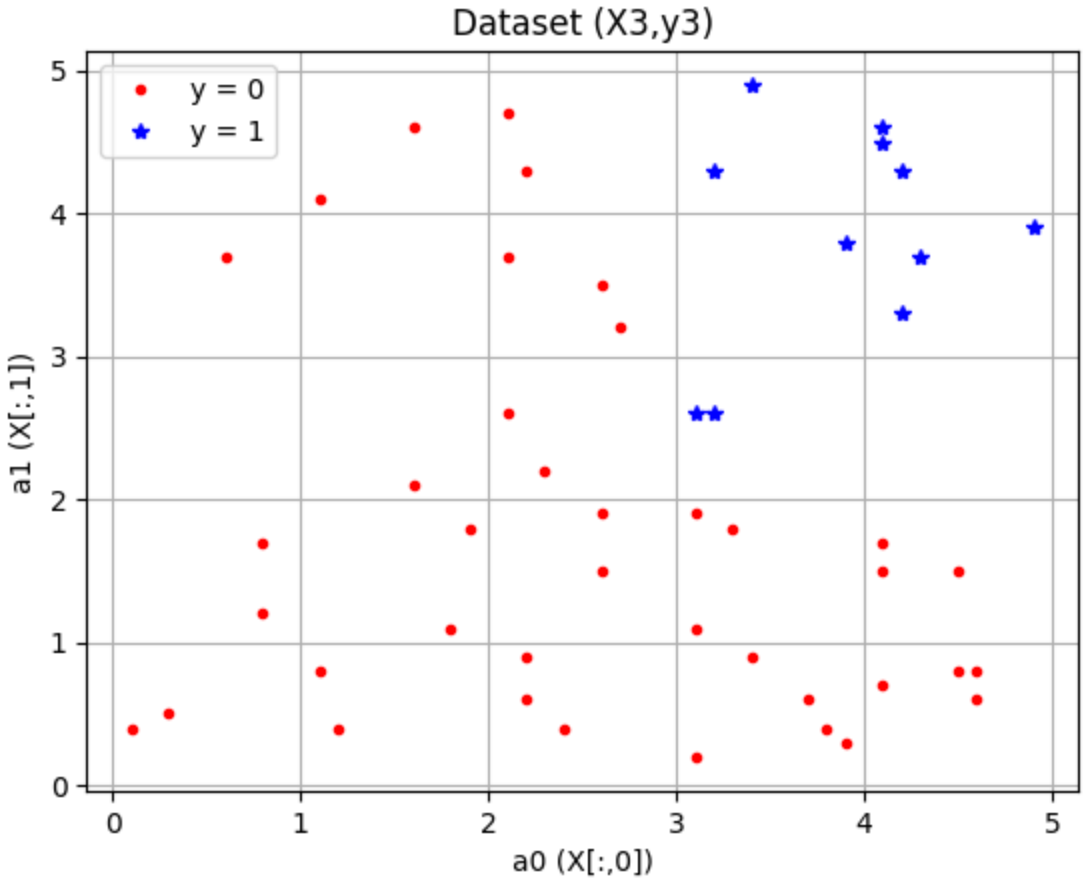
```
Using data from Datasets_xy
```

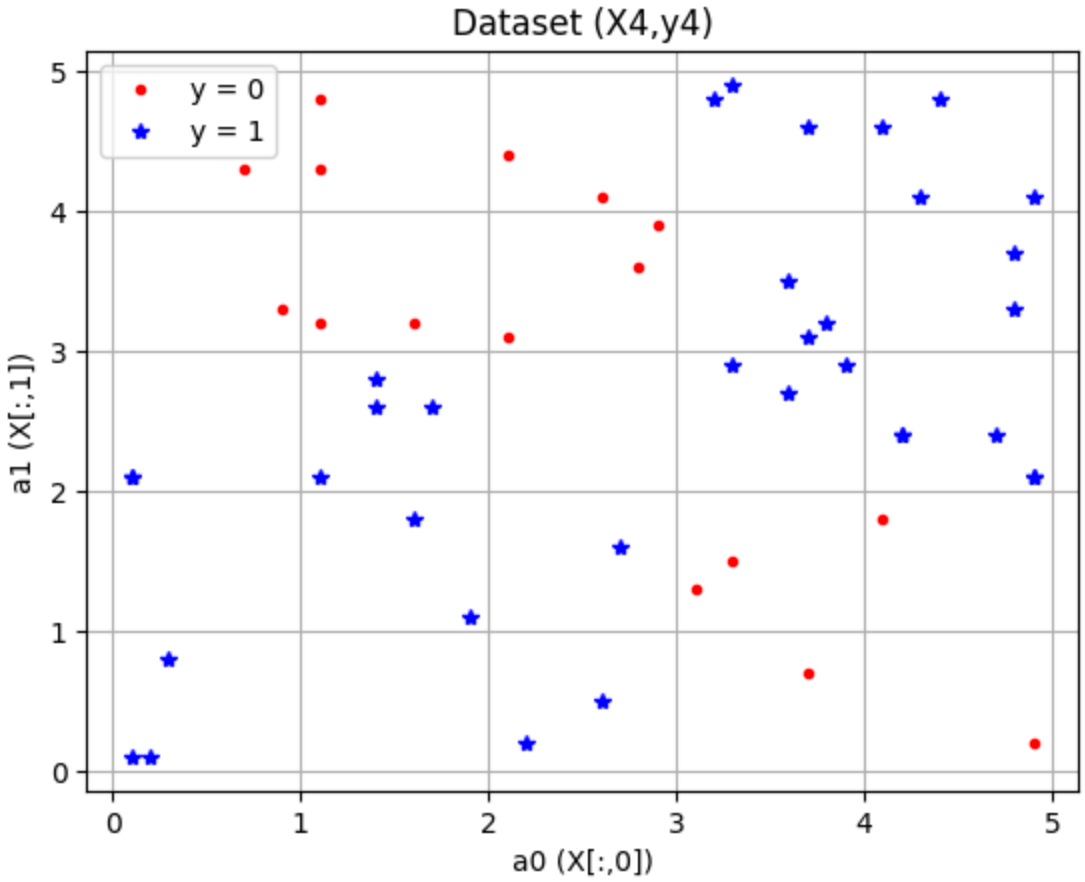
In [14]:
```
marks = ['.r', '*b','+k']
for i in range(7):
  X = np.load(f'{data_dir}/X{i}.npy')
  y = np.load(f'{data_dir}/y{i}.npy')
  plt.figure()
  for c in range(np.max(y)+1):
    ind = y==c
    plt.plot(X[ind,0],X[ind,1],marks[c],label=f'y = {c}')
  plt.legend()
  plt.grid(True)
  plt.xlabel("a0 (X[:,0])")
  plt.ylabel("a1 (X[:,1])")
  plt.title(f'Dataset (X{i},y{i})')
plt.show()
```

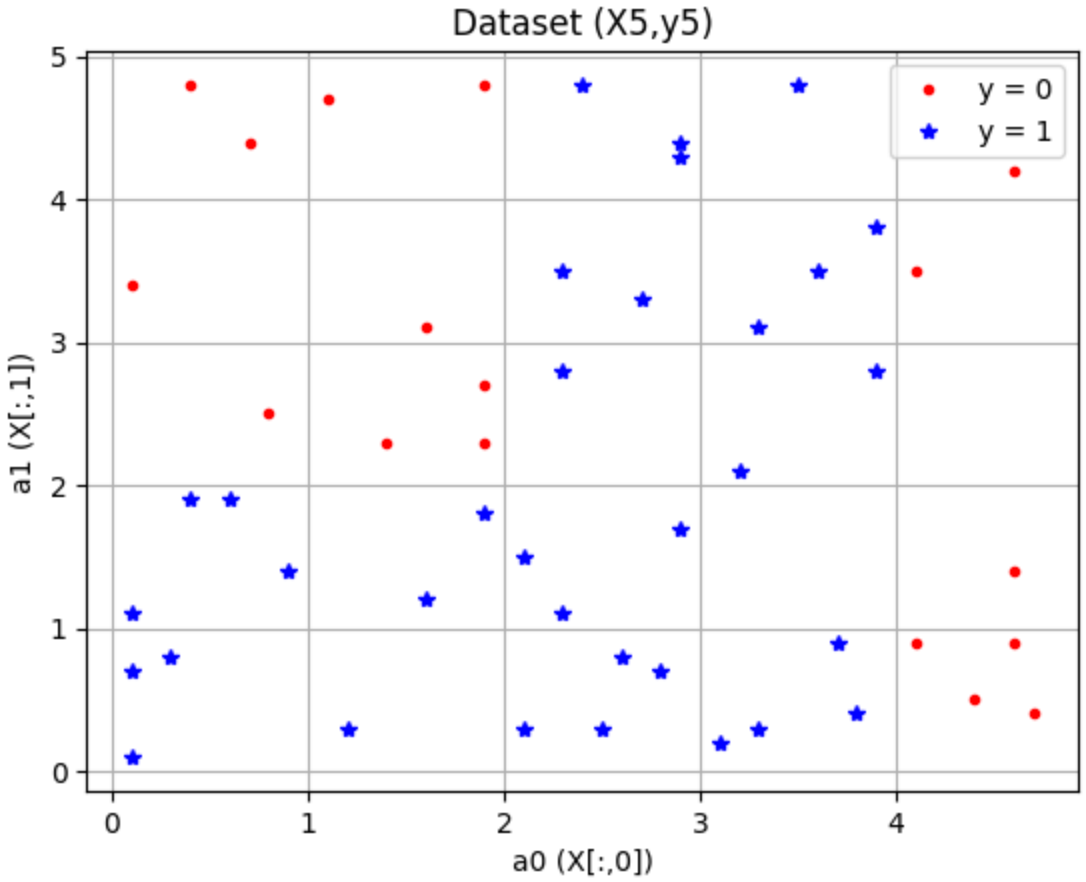

Dataset (X0,y0)
a1 (X[:,1])
a0 (X[:,0])
y = 0
y = 1
0
1
2
3
4
5

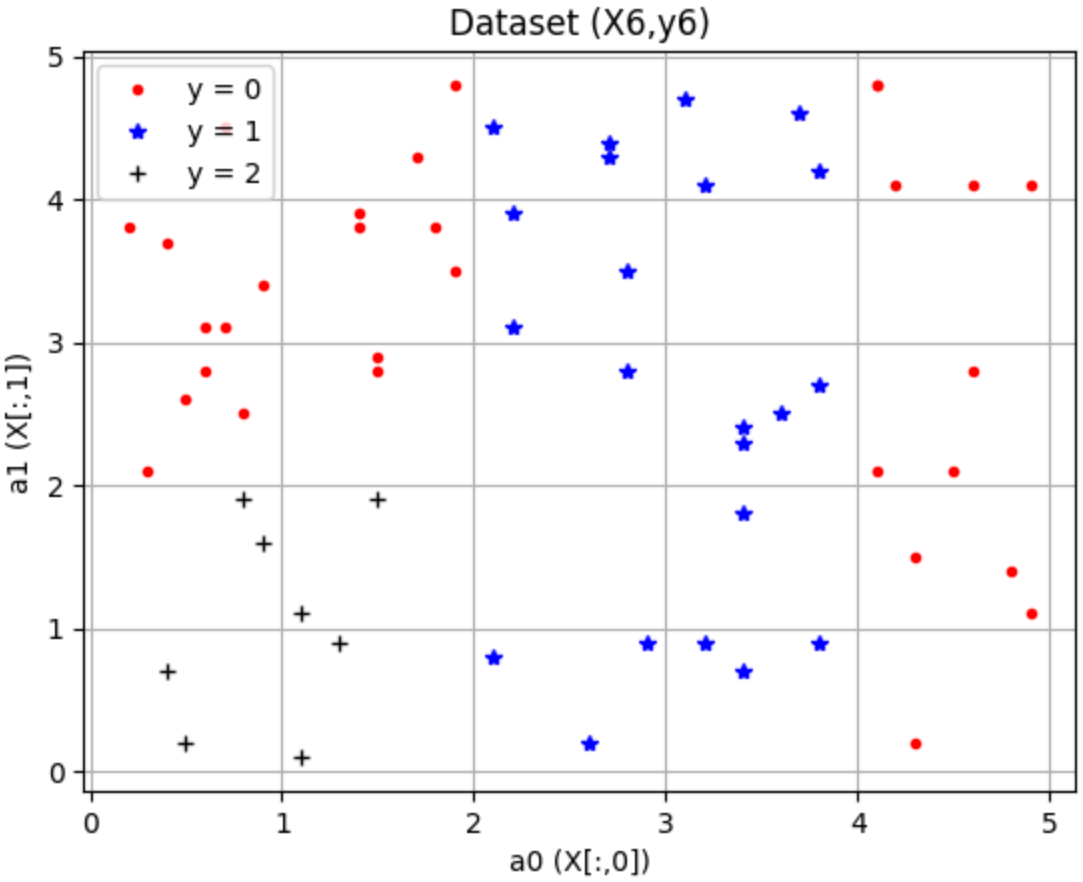

Dataset (X1,y1)
y = 0
y = 1
a1 (X[:,1])
a0 (X[:,0])
0
1
2
3
4
5


Dataset (X2,y2)
y = 0
y = 1
a1 (X[:,1])
a0 (X[:,0])
0
1
2
3
4
5


Dataset (X3,y3)
y = 0
y = 1
a1 (X[:,1])
a0 (X[:,0])
0
1
2
3
4
5


Dataset (X4,y4)
y = 0
y = 1
a1 (X[:,1])
a0 (X[:,0])
0
1
2
3
4
5


Dataset (X5,y5)
y = 0
y = 1
a1 (X[:,1])
a0 (X[:,0])
0
1
2
3
4
5



Exercise

For each of the datasets *(X0,y0), ..., (X6,y6)* write a function that would predict y with perfect accuracy. Use only *if* statements of the form $(a \leq t)$ where a is an attribute and t is a threshold.

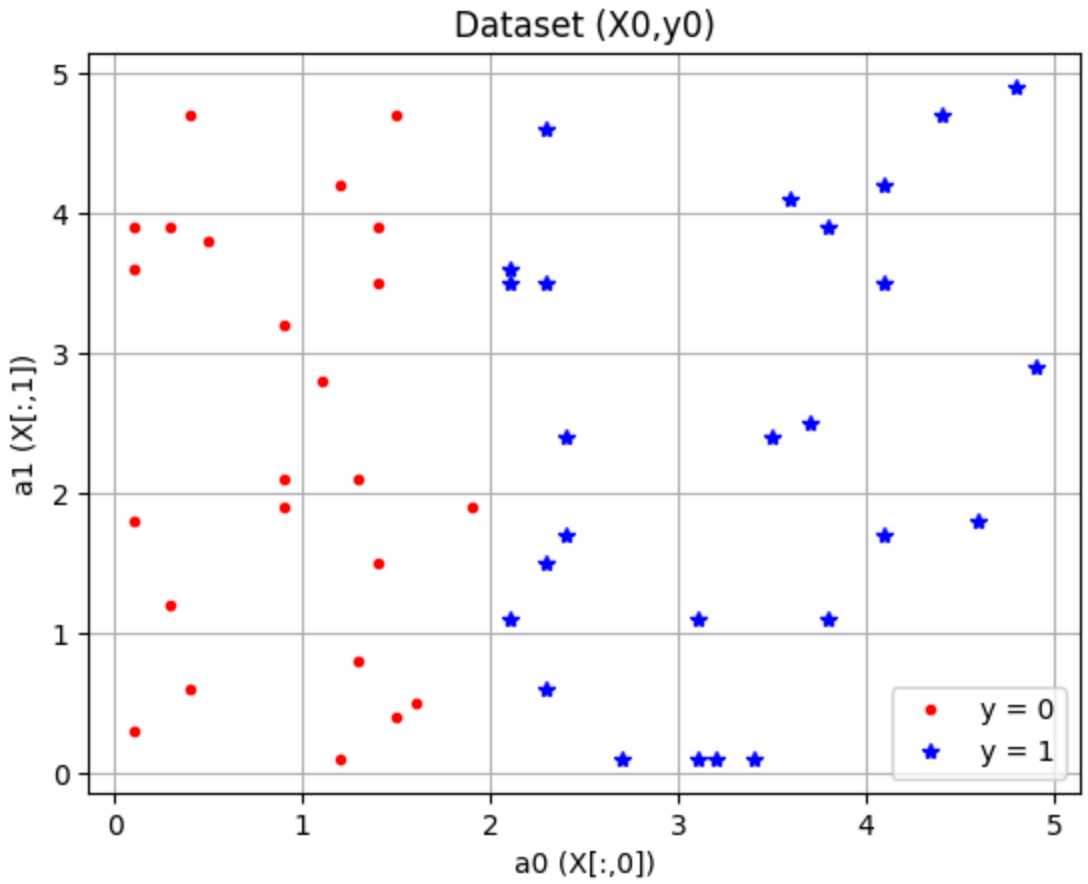
Dataset (X0,y0)

In [15]:
```
X = np.load(f'{data_dir}/X0.npy')
y = np.load(f'{data_dir}/y0.npy')
plt.figure()
```

```
for c in range(np.max(y)+1):
  ind = y==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'y = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
plt.title(f'Dataset (X0,y0)')
plt.show()

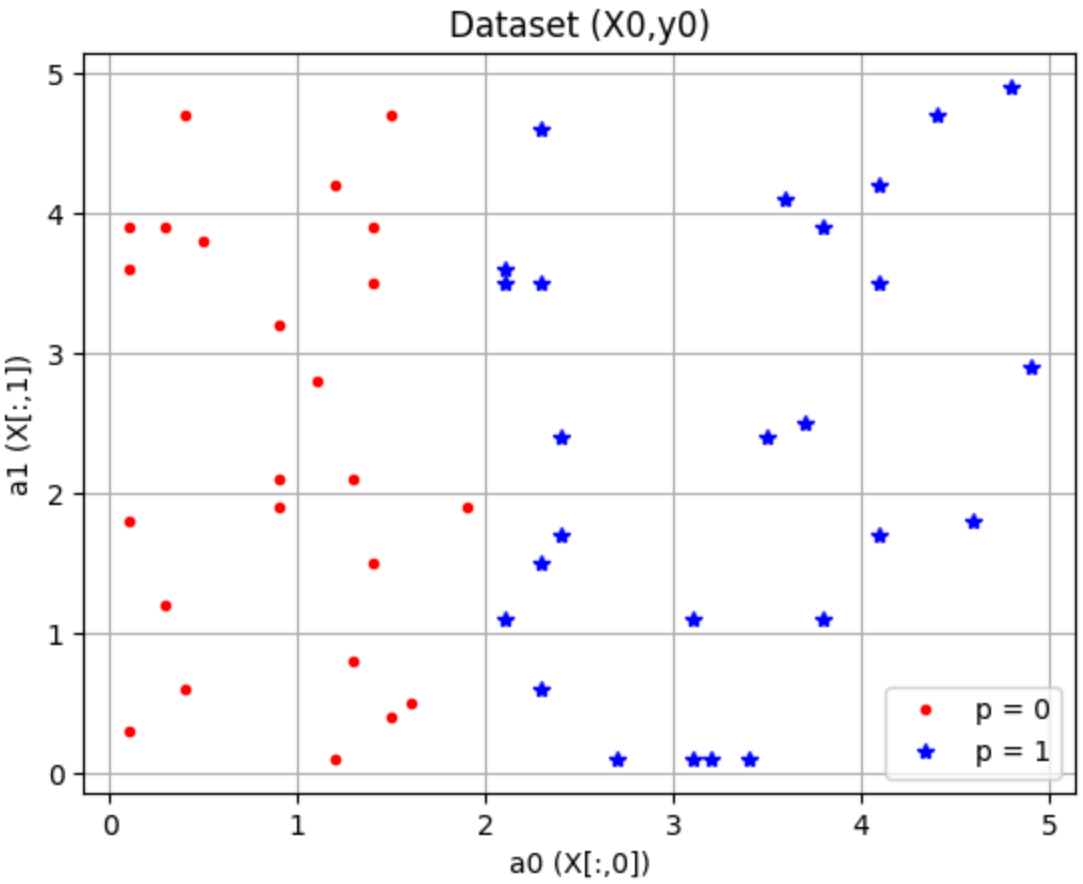
def pred(X):
  p = np.zeros(X.shape[0],dtype=int)
  for i in range(len(p)):
    if X[i,0] <= 2:  # a0 <= 2 -> class 0, else class 1
      p[i] = 0
    else:
      p[i] = 1
  return p

p = pred(X)
print(f'{accuracy_score(p,y) = :.2f}')
```




```
accuracy_score(p,y) = 1.00
```

In [16]:
```
plt.figure()
for c in range(np.max(y)+1):
  ind = p==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'p = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
plt.title(f'Dataset (X0,y0)')
plt.show()
```



Dataset (X1,y1)

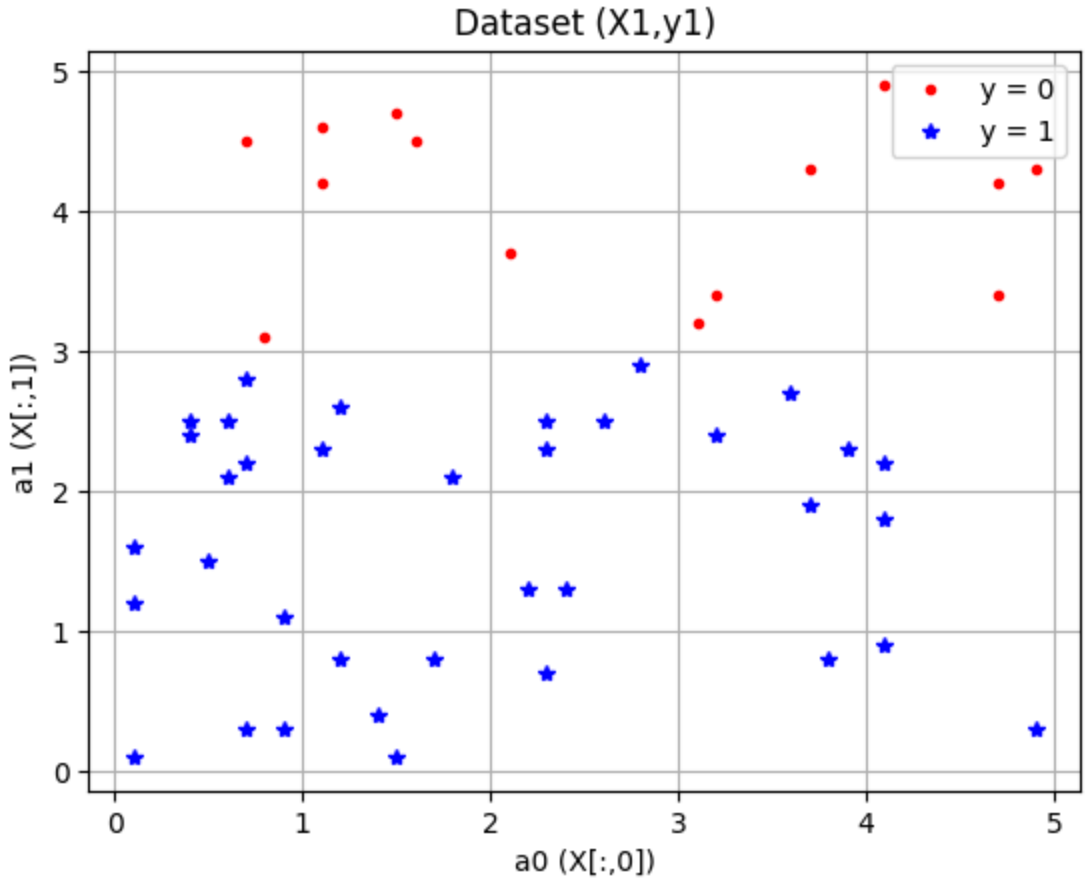
In [17]:
```
X = np.load(f'{data_dir}/X1.npy')
y = np.load(f'{data_dir}/y1.npy')
plt.figure()
for c in range(np.max(y)+1):
  ind = y==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'y = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
```

```
plt.title(f'Dataset (X1,y1)')
plt.show()

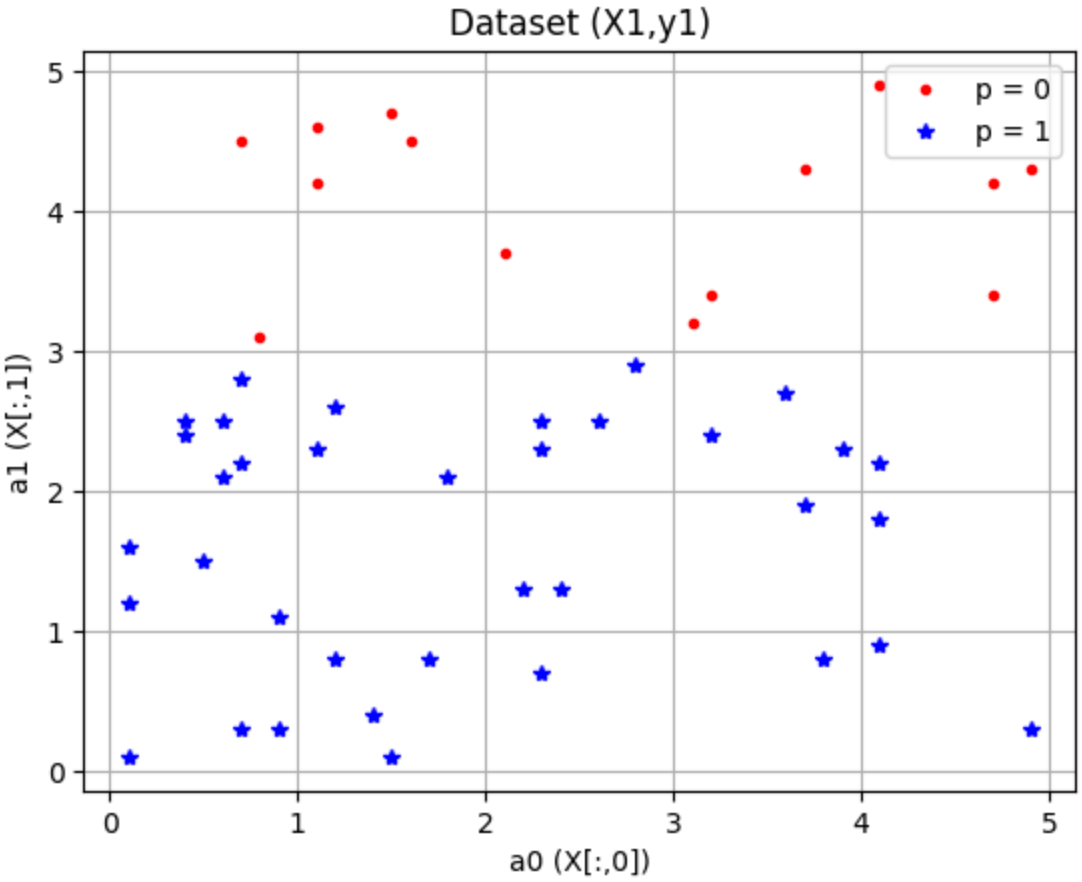
# Dataset 1: Class 0 has a1 high [3.1-4.9], Class 1 has a1 low [0.1-2.9]. a1 <= 3 -> class 1.
def pred(X):
  p = np.zeros(X.shape[0],dtype=int)
  for i in range(len(p)):
    if X[i,1] <= 3:  # a1 <= 3 -> class 1, else class 0
      p[i] = 1
    else:
      p[i] = 0
  return p

p = pred(X)
print(f'{accuracy_score(p,y) = :.2f}')

# Plot predictions
plt.figure()
for c in range(np.max(y)+1):
  ind = p==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'p = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
plt.title(f'Dataset (X1,y1)')
plt.show()
```




```
accuracy_score(p,y) = 1.00
```



Dataset (X2,y2)

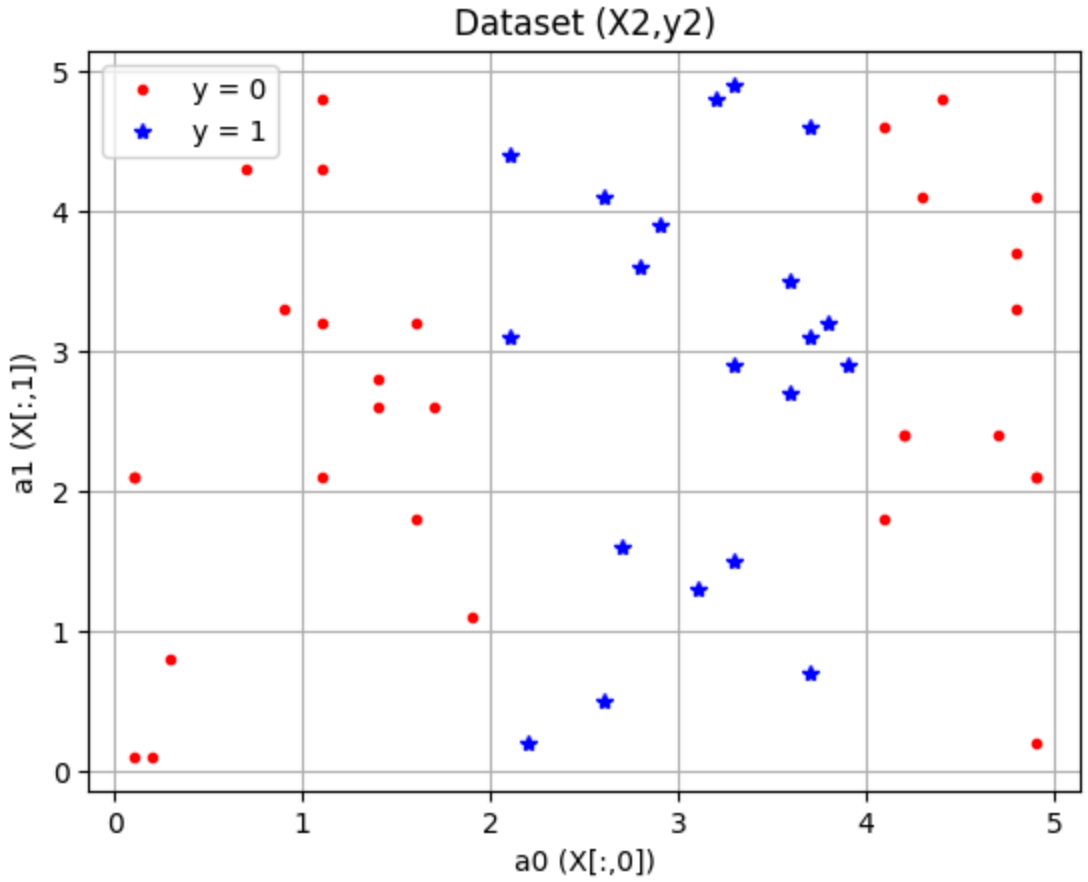
In [18]:
```
X = np.load(f'{data_dir}/X2.npy')
y = np.load(f'{data_dir}/y2.npy')
plt.figure()
for c in range(np.max(y)+1):
  ind = y==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'y = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
```

```
plt.title('Dataset (X2,y2)')
plt.show()

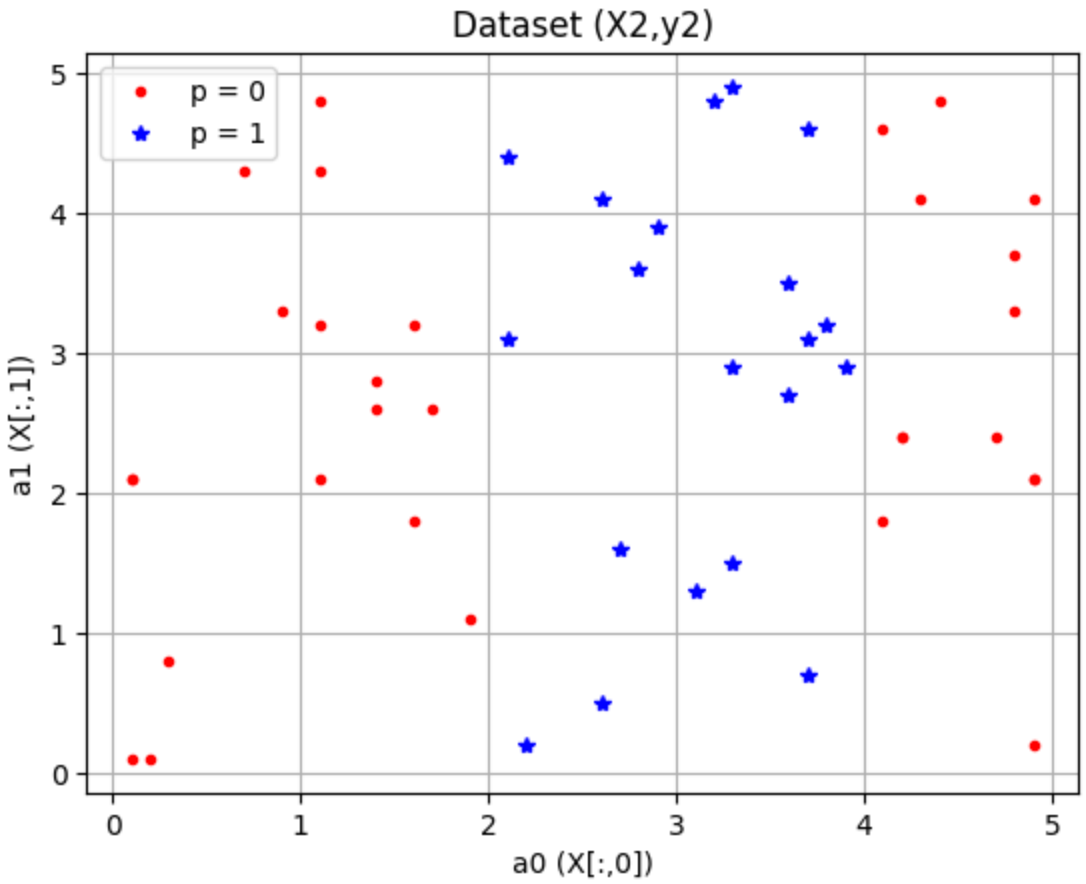
# Dataset 2: Class 1 is in middle strip a0 [2.1, 3.9]. a0 <= 2 -> 0, a0 <= 4 -> 1, else 0.
def pred(X):
  p = np.zeros(X.shape[0],dtype=int)
  for i in range(len(p)):
    if X[i,0] <= 2:
      p[i] = 0
    elif X[i,0] <= 4:
      p[i] = 1
    else:
      p[i] = 0
  return p

p = pred(X)
print(f'{accuracy_score(p,y) = :.2f}')

plt.figure()
for c in range(np.max(y)+1):
  ind = p==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'p = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
plt.title('Dataset (X2,y2)')
plt.show()
```




```
accuracy_score(p,y) = 1.00
```



Dataset (X3,y3)

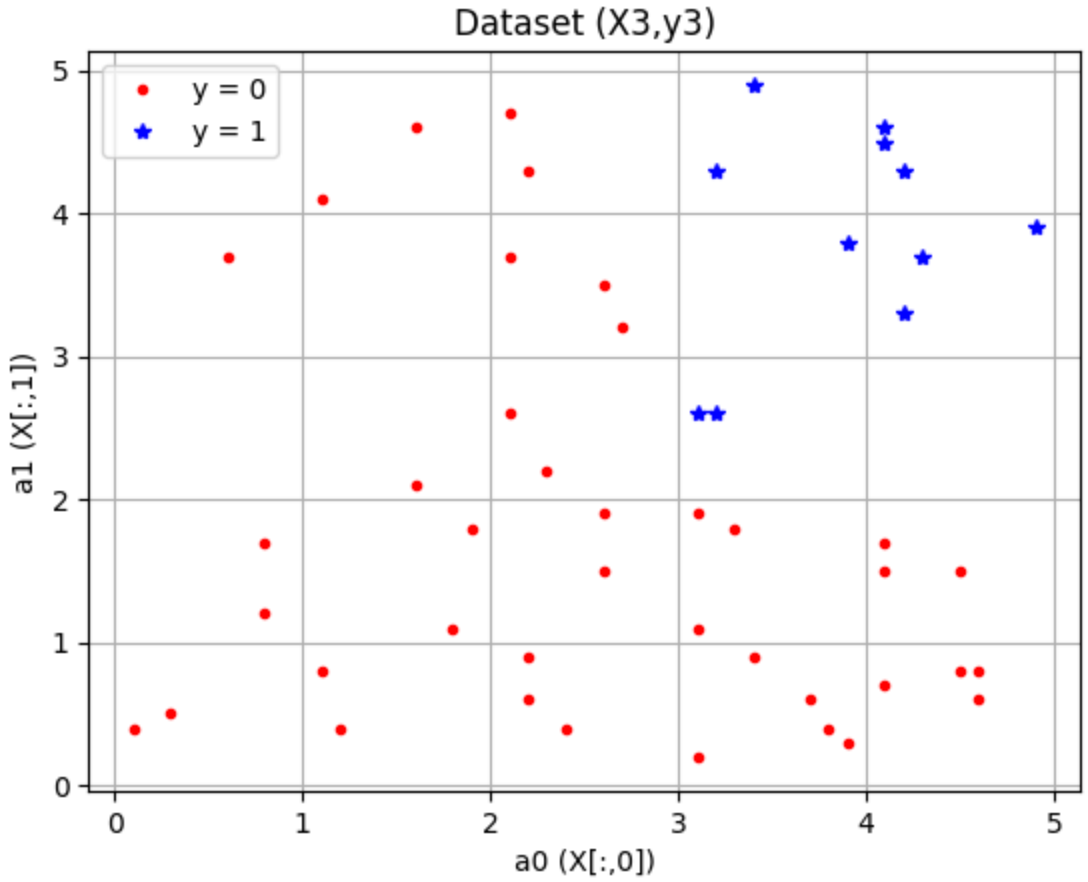
In [19]:
```
X = np.load(f'{data_dir}/X3.npy')
y = np.load(f'{data_dir}/y3.npy')
plt.figure()
for c in range(np.max(y)+1):
  ind = y==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'y = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
```

```
plt.title('Dataset (X3,y3)')
plt.show()

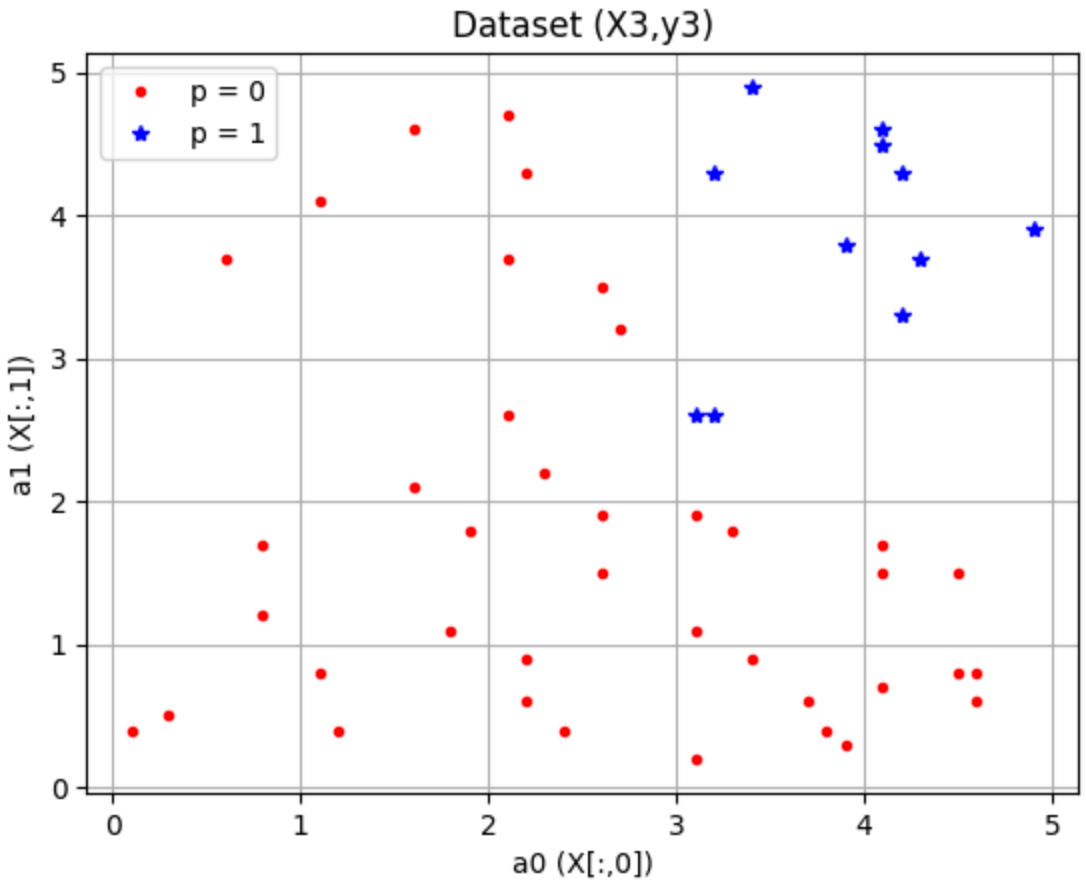
# Dataset 3: Class 1 is top-right (a0>=3.1, a1>=2.6). if a0<=3: 0, elif a1<=2.5: 0, else: 1
def pred(X):
  p = np.zeros(X.shape[0],dtype=int)
  for i in range(len(p)):
    if X[i,0] <= 3:
      p[i] = 0
    elif X[i,1] <= 2.5:
      p[i] = 0
    else:
      p[i] = 1
  return p

p = pred(X)
print(f'{accuracy_score(p,y) = :.2f}')

plt.figure()
for c in range(np.max(y)+1):
  ind = p==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'p = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
plt.title('Dataset (X3,y3)')
plt.show()
```




```
accuracy_score(p,y) = 1.00
```



Dataset (X4,y4)

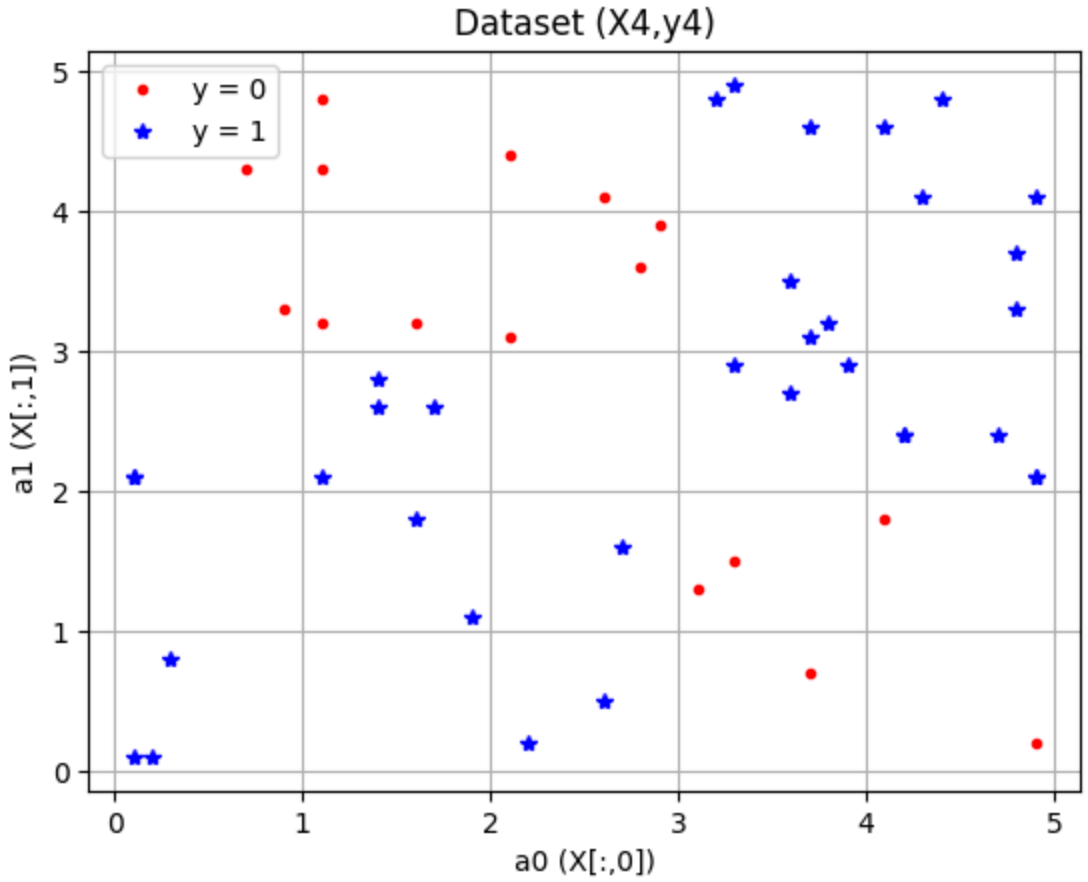
In [20]:
```
X = np.load(f'{data_dir}/X4.npy')
y = np.load(f'{data_dir}/y4.npy')
plt.figure()
for c in range(np.max(y)+1):
  ind = y==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'y = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
```

```
plt.title('Dataset (X4,y4)')
plt.show()

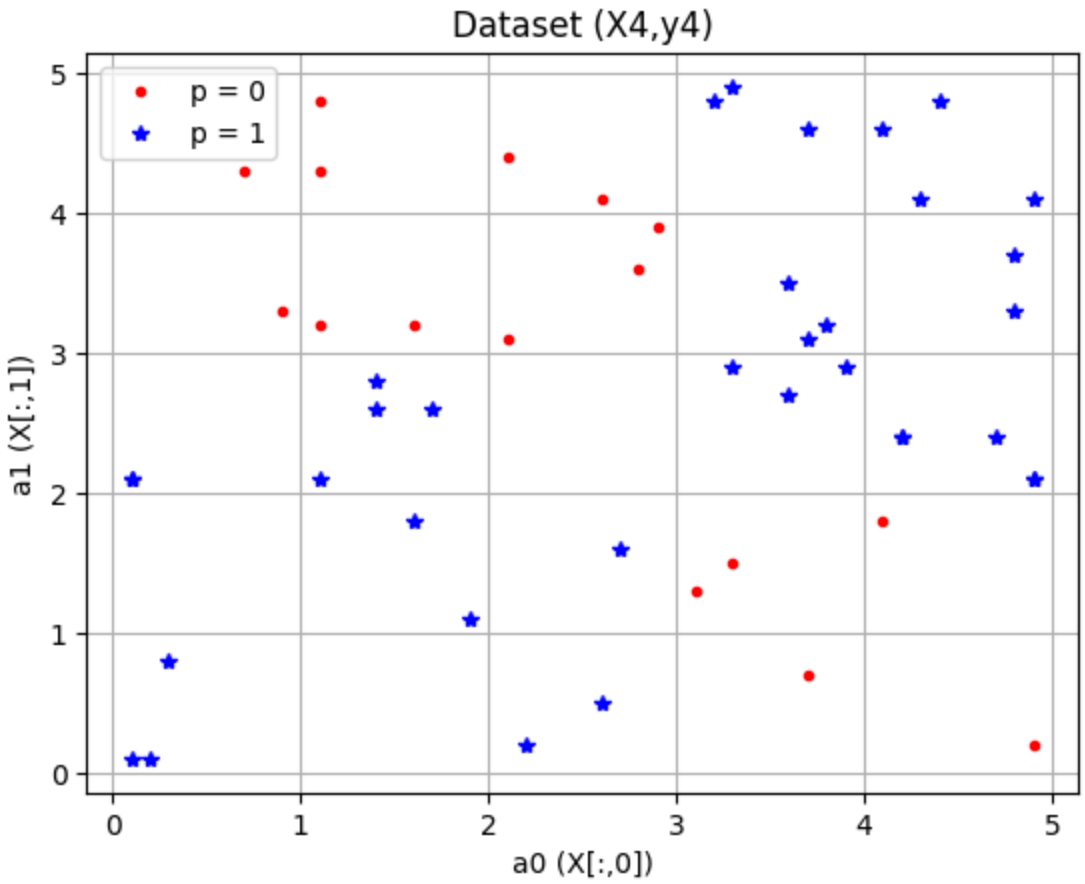
# Dataset 4: Complex - needs 4 (a<=t) rules. a0<=3: if a1<=3 then a0<=2.9->1 else 0; else: a1<=2->0 else 1
def pred(X):
  p = np.zeros(X.shape[0],dtype=int)
  for i in range(len(p)):
    if X[i,0] <= 3:
      if X[i,1] <= 3:
        p[i] = 1 if X[i,0] <= 2.9 else 0
      else:
        p[i] = 0
    else:
      p[i] = 0 if X[i,1] <= 2 else 1
  return p

p = pred(X)
print(f'{accuracy_score(p,y) = :.2f}')

plt.figure()
for c in range(np.max(y)+1):
  ind = p==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'p = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
plt.title('Dataset (X4,y4)')
plt.show()
```




```
accuracy_score(p,y) = 1.00
```



Dataset (X5,y5)

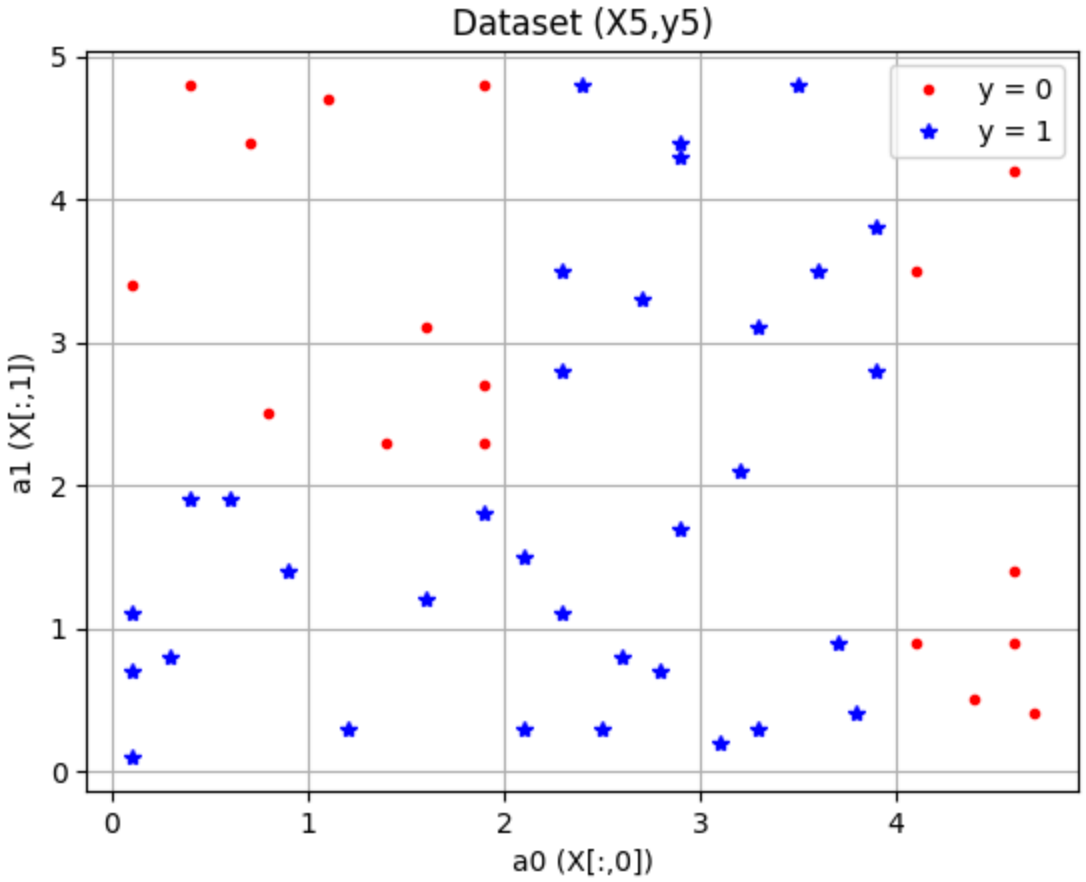
In [21]:
```
X = np.load(f'{data_dir}/X5.npy')
y = np.load(f'{data_dir}/y5.npy')
plt.figure()
for c in range(np.max(y)+1):
  ind = y==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'y = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
```

```
plt.title('Dataset (X5,y5)')
plt.show()

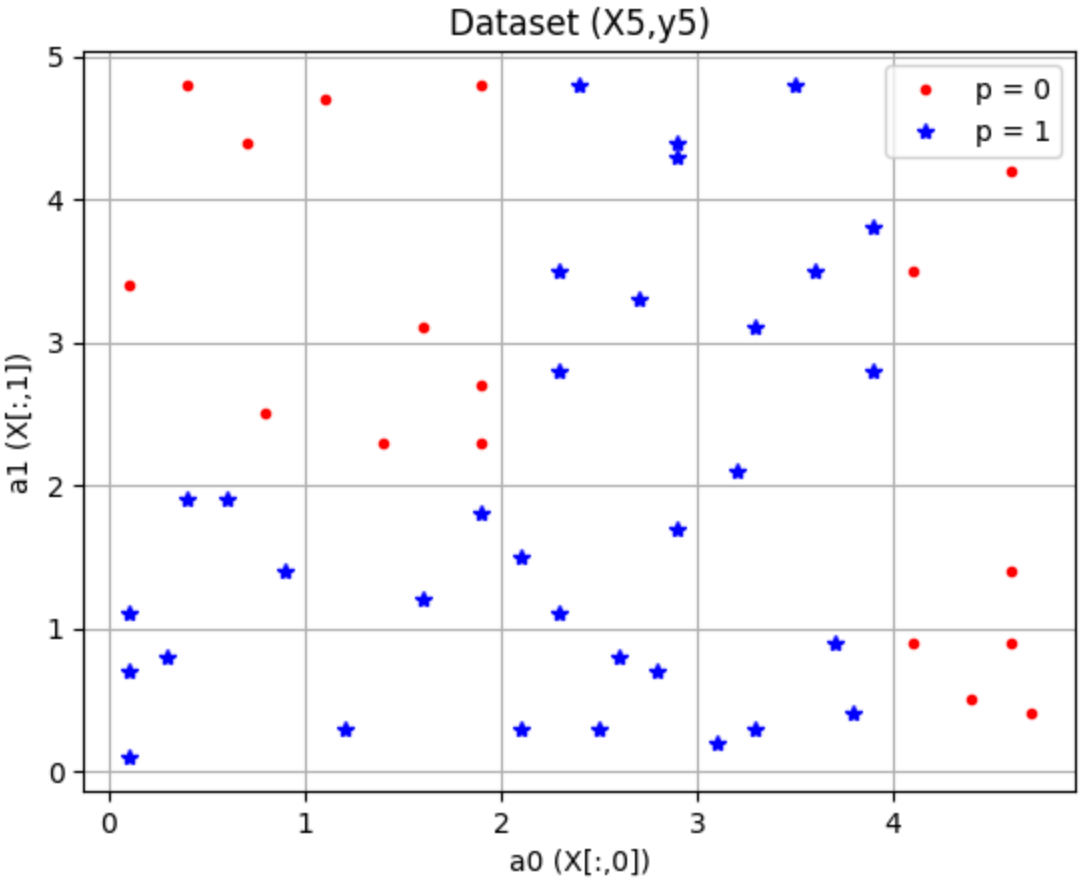
# Dataset 5: a0>4->0; a0<=4 and a1<=2.2->1; a0<=4 and a1>2.2 and a0<=2->0; else 1
def pred(X):
  p = np.zeros(X.shape[0],dtype=int)
  for i in range(len(p)):
    if X[i,0] <= 4:
      if X[i,1] <= 2.2:
        p[i] = 1
      else:
        p[i] = 0 if X[i,0] <= 2 else 1
    else:
      p[i] = 0
  return p

p = pred(X)
print(f'{accuracy_score(p,y) = :.2f}')

plt.figure()
for c in range(np.max(y)+1):
  ind = p==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'p = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
plt.title('Dataset (X5,y5)')
plt.show()
```




```
accuracy_score(p,y) = 1.00
```



Dataset (X6,y6) - 3 classes

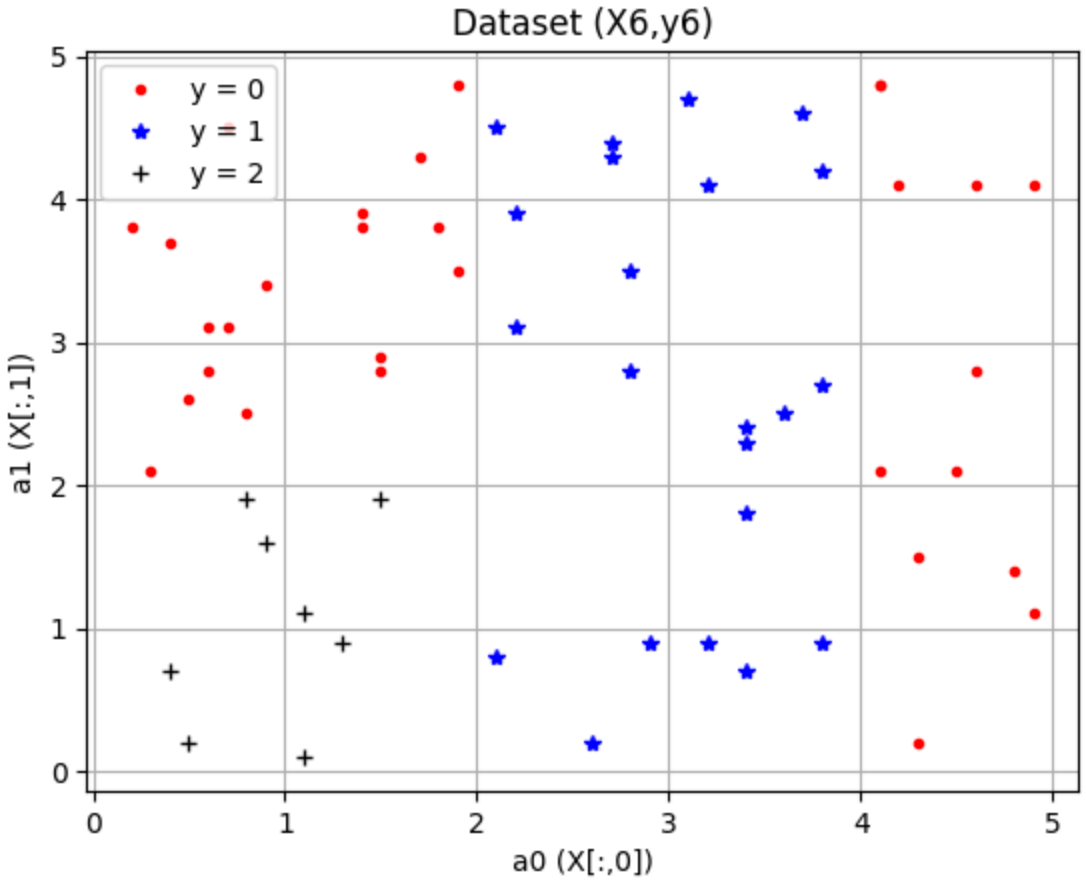
In [22]:
```
X = np.load(f'{data_dir}/X6.npy')
y = np.load(f'{data_dir}/y6.npy')
plt.figure()
for c in range(np.max(y)+1):
  ind = y==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'y = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
```

```
plt.title('Dataset (X6,y6)')
plt.show()

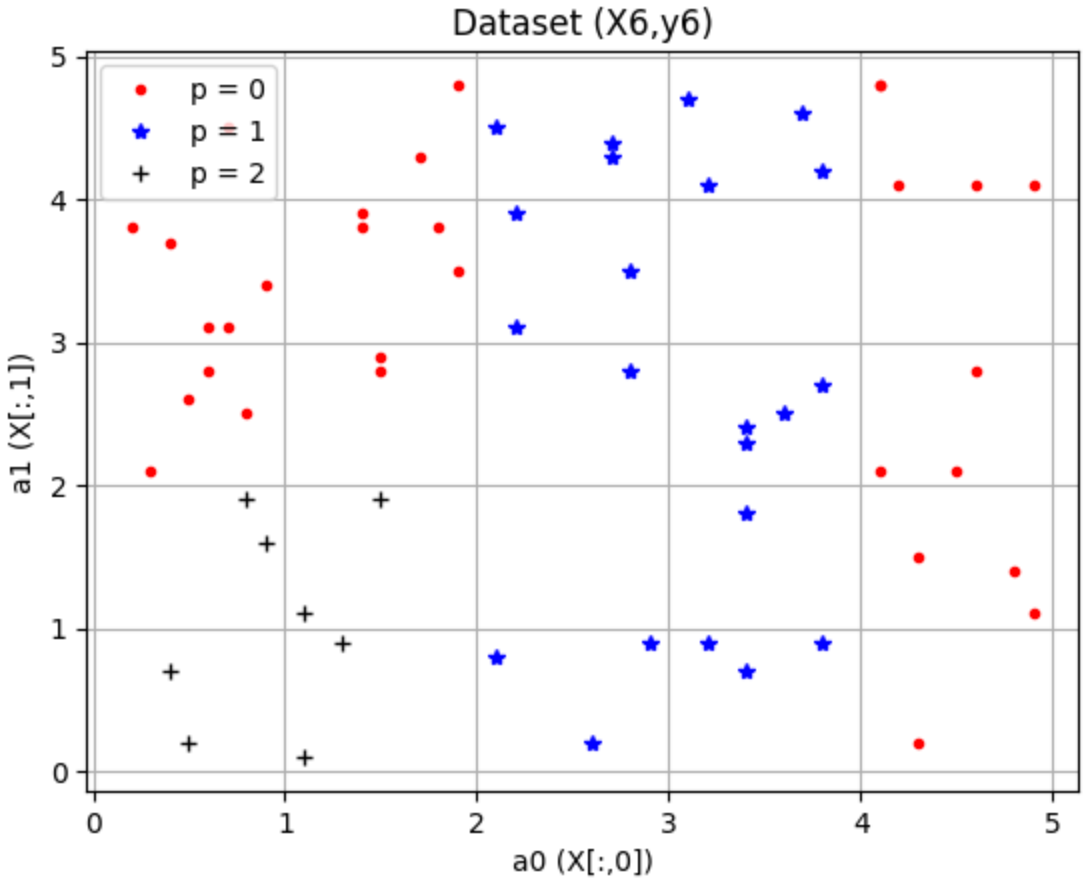
# Dataset 6: 3 classes. Class 2: bottom-left (a1<=2, a0<=1.5). Class 1: middle band (2<a0<=4). Class 0: rest.
# When a1<=2: a0<=1.5->2, a0<=4->1, else 0. When a1>2: a0<=2->0, a0<=4->1, else 0.
def pred(X):
  p = np.zeros(X.shape[0],dtype=int)
  for i in range(len(p)):
    if X[i,1] <= 2:
      if X[i,0] <= 1.5:
        p[i] = 2
      elif X[i,0] <= 4:
        p[i] = 1
      else:
        p[i] = 0
    else:
      if X[i,0] <= 2:
        p[i] = 0
      elif X[i,0] <= 4:
        p[i] = 1
      else:
        p[i] = 0
  return p

p = pred(X)
print(f'{accuracy_score(p,y) = :.2f}')

plt.figure()
for c in range(np.max(y)+1):
  ind = p==c
  plt.plot(X[ind,0],X[ind,1],marks[c],label=f'p = {c}')
plt.legend()
plt.grid(True)
plt.xlabel("a0 (X[:,0])")
plt.ylabel("a1 (X[:,1])")
plt.title('Dataset (X6,y6)')
plt.show()
```




```
accuracy_score(p,y) = 1.00
```


Dataset (X6,y6)
p = 0
p = 1
p = 2
a1 (X[:,1])
a0 (X[:,0])
0
1
2
3
4
5